Secure Private Data Vault: AES / RC4 / DES file storage and AES-mode benchmark

1. Title and assignment information

Course / assignment: Information Security, Assignment 1 (group of four)
Group members: *[Name 1 – Student ID], [Name 2 – Student ID], [Name 3 – Student ID], [Name 4 – Student ID]*
Lecturer / submission date: *[to be filled in]*
Repository contents: Flask application, `vault_crypto/` cipher module, `benchmark/`, `tests/`, this report.

2. Objectives and requirements

The assignment slides require a web application that (a) stores users' private data in a database, taking GDPR / UU PDP as the reference for what is private; (b) stores an ID-card image, PDF/DOC/XLS files and video files; (c) encrypts all stored data with AES, RC4 and DES; (d) lets users retrieve their decrypted data; and (e) requires a username/password. The second slide adds: use a **non-ECB block-cipher mode** (CBC, CFB, OFB or CTR), any cryptography library, a bug-free program, an analysis of running time and ciphertext differences between the ciphers (downloading the data several times and measuring), a short report, and justification of all decisions. This project covers all four modes (plus AES-GCM as an extra) and keeps RC4 and DES as the assignment demands, labelled obsolete.

3. System architecture and technology stack

Layer	Choice	Why
Web framework	Flask (application factory in `app.py`)	Already used by the original project; small and easy to audit.
Database	SQLite via stdlib `sqlite3`	Original used Flask-SQLAlchemy; replaced by the stdlib module because it needs no extra dependency, uses parameterised SQL, and BLOB columns hold the ciphertext.
Cryptography	`cryptography` (pyca) 46.0.6 on OpenSSL	Maintained, exposes CBC/CFB/OFB/CTR/GCM uniformly, correct PKCS#7, hardware AES where

Layer	Choice	Why
		available. The original used PyCryptodome; both are acceptable under the slide.
Password hashing	Werkzeug `generate_password_hash`	Salted, adaptive (scrypt by default in the installed version); same call as the original.
UI	Original Bootstrap templates, extended	Preserved look; added CSRF fields, algorithm choices, delete, metadata.
Tests	stdlib `unittest` (pytest-compatible)	`pytest` was not installable in the offline build environment.

Request flow: browser → Flask route (session + CSRF check) → `uploads.validate_upload` → `vault_crypto.encrypt_bytes` → SQLite BLOB. Download: ownership-scoped SQL lookup → `decrypt_bytes` (verify, then decrypt) → bytes streamed from memory to the browser.

4. Database design and storage decisions

Decision: encrypted bytes are stored **inside SQLite** (`BLOB`), not on the file system. The original stored JSON envelopes under `ciphertexts/` and kept only a path in the DB, which weakens the claim "stored in a database" and also created path-handling risks. Table `encrypted_files` holds: `user_id` (foreign key, ownership), `original_filename`, `category` (`id_card/document/video`), `mime_type`, `algorithm` (e.g. `AES-CBC`), `kdf`, `kdf_iterations`, `salt`, `iv`, `mac`, `plaintext_size`, `ciphertext_size`, `ciphertext`, `uploaded_at`, `group_id`. `users` holds only `username`, the password hash and the creation time. No plaintext file content is stored or written to disk (the original wrote decrypted files into `uploads/`).

Trade-off: SQLite BLOBs are simple and transactional, but a whole file is held in memory while processing, so the default limit is 100 MB (`MAX_UPLOAD_MB`). Multi-gigabyte video would need chunked/streaming encryption or external object storage; this is a documented limitation. **Needs lecturer confirmation:** whether "stored in a database" accepts ciphertext as BLOB (what we did) or only a storage reference. Filenames, categories and sizes are plaintext metadata (they can themselves be personal data); encrypting them is future work.

Privacy classification. GDPR Art. 4(1) defines personal data broadly; Art. 9 adds special categories (e.g. health data, biometric data used to uniquely identify a person). UU PDP No. 27/2022 Art. 4 distinguishes *specific* personal data (health, biometric, genetic, criminal records, children's data, personal financial data) from *general* personal data (full name, gender, nationality, religion, marital status, and data combined to identify a person). An ID-card image contains general identifiers and a facial photo and should be treated as high-risk; documents and videos may contain any category. The app therefore treats **every upload as sensitive**,

minimises what it stores about the user (no e-mail, no real name field), and lets the user delete files.

5. Authentication and authorization

- Registration: username `[A-Za-z0-9_.-]{3,32}` (case-insensitive unique), password ≥ 8 characters; stored with Werkzeug salted hashing. Login uses a constant-work check for unknown users and a per-(username, IP) throttle (5 failures / 5 min → HTTP 429).
- Sessions: `SECRET_KEY` is **required from the environment** (the original generated a random key at each start, invalidating sessions and breaking multi-worker use); cookies are `HttpOnly`, `SameSite=Lax`, `Secure` by default (set `SESSION_COOKIE_SECURE=0` only for plain-http localhost), 30-minute lifetime; the session is replaced on login.
- CSRF: every state-changing request (including logout, which is now POST) must carry a per-session token compared with `hmac.compare_digest`.
- Authorization: every file query is `WHERE id = ? AND user_id = ?`; another user's file simply does not exist (HTTP 404, no enumeration). This applies to download/decrypt, delete and the metadata endpoint (`/files/<id>/meta`, which never returns key material or ciphertext).
- Hardening: `nosniff`, `X-Frame-Options: DENY`, CSP, `Referrer-Policy: no-referrer`, `Cache-Control: no-store` for authenticated pages; generic error pages (no stack traces); `debug=False`.

6. Encryption and decryption workflows

Upload. Validate (category, extension whitelist, declared MIME, magic bytes, non-empty, size limit, sanitised filename) → derive keys → encrypt → store. **Key schedule.** `PBKDF2-HMAC-SHA256(password, random 16-byte salt, 600000 iterations) → 32-byte master → HKDF-Expand` with distinct labels gives an **encryption key** (32 B; 8 B for DES) and a separate **MAC key**. Salt (per file), IV/nonce (per encryption) are random from `os.urandom`, stored beside the ciphertext, and are public values. The per-file password is chosen by the user at upload time and is never stored. **Integrity.** AES-CBC/CFB/OFB/CTR, DES and RC4 give confidentiality only, so the envelope adds **encrypt-then-MAC**: HMAC-SHA256 over a length-prefixed header (algorithm, iterations, salt, IV) and the ciphertext; it is verified in constant time **before** any decryption. AES-GCM authenticates itself (header as AAD).
Download. Ownership check → verify → decrypt → compare length with stored `plaintext_size` → stream. A wrong password, a flipped bit, truncated data or altered metadata all produce the same generic message and **no file**. This fixes the original behaviour, in which CBC/RC4 could return corrupted plaintext as a "successful" download, and its `pkcs7unpad` accepted any padding byte.

7. AES, RC4 and DES implementation decisions

Algorithm	Implementation	Notes
AES-256-CBC (app default)	PKCS#7 via the library's	Non-ECB mode required by

Algorithm	Implementation	Notes
	padder/unpadder	the slide.
AES-256-CFB (128-bit segments), OFB, CTR	no padding (stream-like)	CTR uses a random 16-byte initial counter block; never reuse key+nonce.
AES-256-GCM	12-byte nonce, 16-byte tag	Extra, modern, authenticated option.
DES-CBC	3DES primitive with an 8-byte key (K1=K2=K3), identical to single DES; PKCS#7	The library no longer exposes single DES. **56-bit key: brute-forceable; 64-bit blocks.**
RC4	`ARC4` with a 32-byte derived key	A **stream cipher**, not a block-cipher mode; biased keystream, prohibited in TLS (RFC 7465).

RC4 and DES are included **only** to satisfy the assignment and are labelled as obsolete in the UI. They must not protect real personal data. For the slide's "encrypted with all of these algorithms" the user can choose one algorithm per file or **ALL**, which stores three independent encrypted copies (AES-CBC, DES-CBC, RC4). Alternative readings (e.g. layered encryption) exist: **confirm with the lecturer**.

8. AES mode benchmarking methodology

Library justification: the same `cryptography`/OpenSSL code path is used by the app and the benchmark (`vault_crypto.raw_encrypt/raw_decrypt`), so results describe the shipped implementation. *Inputs (synthetic, deterministic, no personal data; regenerated by `benchmark/generate_fixtures.py`, hashes in `MANIFEST.json`):* random binary at 1 KiB, 64 KiB, 1 MiB, 10 MiB, 50 MiB; 1 MiB seeded text; a 1024x768 PNG; a 200-page PDF built by hand; a 150-frame MJPEG AVI (OpenCV). Every algorithm receives byte-identical plaintext. *Fairness:* same 256-bit key for all AES modes; fresh random IV/nonce each trial generated **outside** the timer; warm-up runs discarded; algorithm order **shuffled each trial** to spread drift and noise evenly; GC disabled inside timed regions; correctness (`decrypt(encrypt(x)) == x`) asserted after every trial outside the timer, so the benchmark would abort on any mismatch. *Timer & regions:* `time.perf_counter_ns` (monotonic). Encryption and decryption are timed **separately** around cipher construction + update + finalisation (+ PKCS#7 pad/unpad for CBC/DES). File read/write is measured separately (Section 10.3), and the assignment's "download" is also measured end-to-end through the Flask app (Section 10.4). *Statistics:* per configuration n = 30: mean, median, sample standard deviation, min, max, coefficient of variation, 95 % CI half-width (1.96·SD/√n), and throughput in MiB/s from the mean (and median) time. *Why 30 trials + 3 warm-ups:* a conventional sample size for the central-limit approximation that keeps the whole run to a few minutes; timings here are noisy (see limits), so more trials would help but cost time. Configurable via `--trials`, `--warmup`. *Modes benchmarked:* AES-CBC, CFB, OFB, CTR (required), AES-GCM (extra AEAD), and DES-CBC

and RC4 as **reference-only legacy** rows (RC4 is not a block-cipher mode and is not compared as one).

9. Test environment and experimental parameters

Item	Value
Python	3.12.3 (CPython)
cryptography	46.0.6
OpenSSL (backend)	OpenSSL 3.5.5 27 Jan 2026
OS	Linux-6.18.44-fc-v153-x86_64-with-glibc2.39
Processor	Intel(R) Xeon(R) Processor @ 2.10GHz
Logical CPUs visible	1
CPU reports AES instructions	True
Timer	time.perf_counter_ns (resolution 1e-09 s, monotonic=True)
Trials / warm-up	30 measured / 3 discarded per file x algorithm x operation
App-download trials	10 per file x algorithm
Run started / finished (UTC)	2026-10-26T09:24:14Z / 2026-10-26T09:28:54Z
Timed region	cipher setup + update + finalize (+PKCS7 pad/unpad); excludes IV generation, file I/O, KDF, verification

Limits: the sandbox is a shared virtual machine exposing **one** logical CPU, so background noise is likely (CV of required-mode timings at 1 and 10 MiB ranged 9 % – 107 %); a few runs show large outliers at 64 KiB. Results are specific to this machine and a single run, and absolute numbers will differ elsewhere. OpenSSL AES-NI usage is reported by the CPU flags but was not independently verified.

10. Measured performance results

All numbers below are produced by `benchmark/run_benchmark.py` and read from `benchmark/results/*.csv` by this report generator. Raw trials: `raw_results.csv` (3780 rows).

10.1 Cipher-only time at 10 MiB and 1 MiB

10 MiB random data

Algorithm	Enc mean ms	Enc SD ms	Enc MiB/s	Dec mean ms	Dec SD ms	Dec MiB/s
AES-CBC	21.09	4.30	474	8.21	1.88	1218

Algorithm	Enc mean ms	Enc SD ms	Enc MiB/s	Dec mean ms	Dec SD ms	Dec MiB/s
AES-CFB	18.60	2.48	538	18.75	2.49	533
AES-OFB	14.40	1.87	694	14.32	2.21	698
AES-CTR	6.32	1.96	1581	5.21	2.19	1920
AES-GCM	9.59	2.88	1043	6.02	2.81	1661
DES-CBC	393.76	28.75	25	369.41	32.04	27
RC4	28.58	2.79	350	29.13	3.40	343

(Medians, min/max, CV and 95 % CI are in `benchmark/results/summary.csv`; ciphertext sizes are in Section 11.)

1 MiB random data

Algorithm	Enc mean ms	Enc SD ms	Enc MiB/s	Dec mean ms	Dec SD ms	Dec MiB/s
AES-CBC	1.30	0.16	769	0.52	0.06	1926
AES-CFB	1.59	0.20	628	1.57	0.20	637
AES-OFB	1.24	0.19	806	1.19	0.19	840
AES-CTR	0.41	0.07	2417	0.35	0.07	2845
AES-GCM	0.46	0.10	2172	0.36	0.06	2777
DES-CBC	34.68	5.04	29	32.57	4.23	31
RC4	2.62	0.27	382	2.53	0.25	395

(Medians, min/max, CV and 95 % CI are in `benchmark/results/summary.csv`; ciphertext sizes are in Section 11.)

10.2 Throughput versus input size (mean MiB/s)

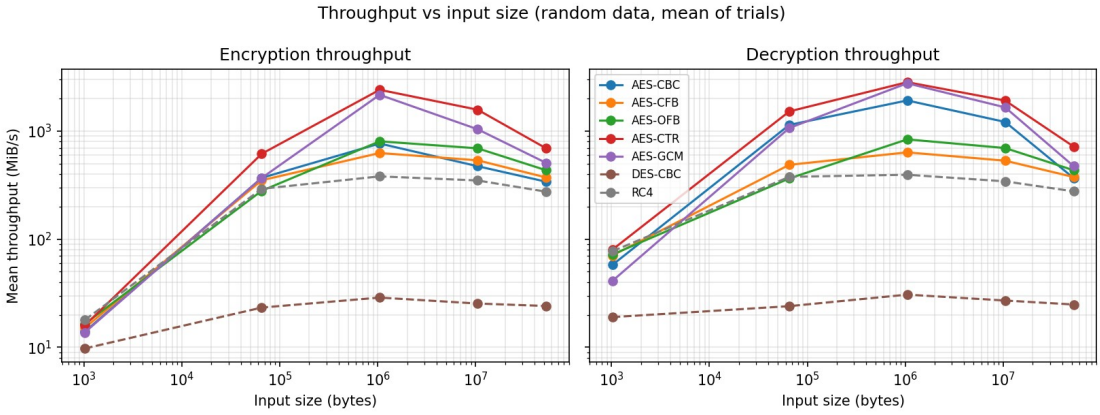
Encryption

Mean throughput (MiB/s)	1 KiB	64 KiB	1 MiB	10 MiB	50 MiB
AES-CBC	14	370	769	474	341
AES-CFB	15	351	628	538	373
AES-OFB	16	281	806	694	435
AES-CTR	16	618	2417	1581	695
AES-GCM	14	369	2172	1043	509
DES-CBC	10	23	29	25	24
RC4	18	293	382	350	276
fastest	**AES-OFB**	**AES-CTR**	**AES-CTR**	**AES-CTR**	**AES-CTR**

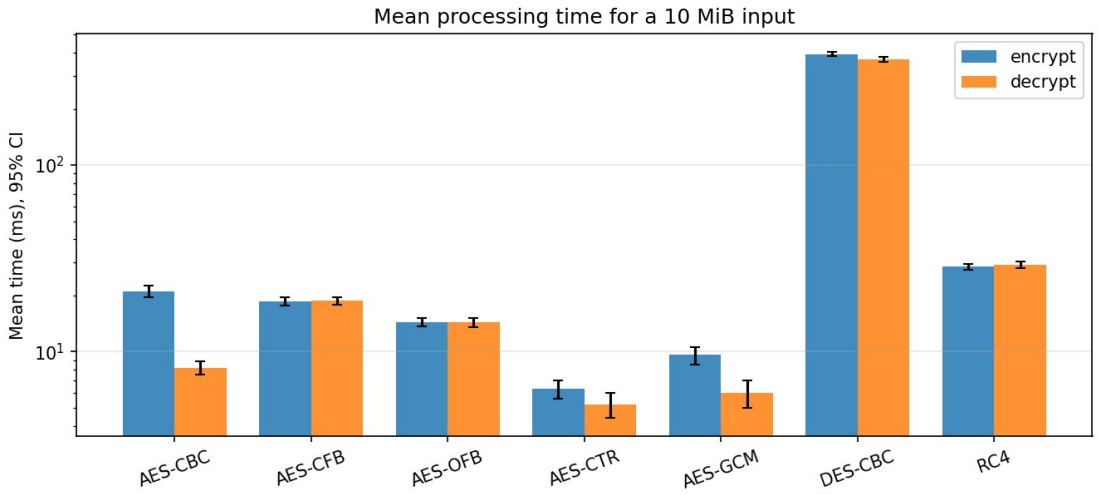
Mean throughput (MiB/s)	1 KiB	64 KiB	1 MiB	10 MiB	50 MiB
required mode					

Decryption

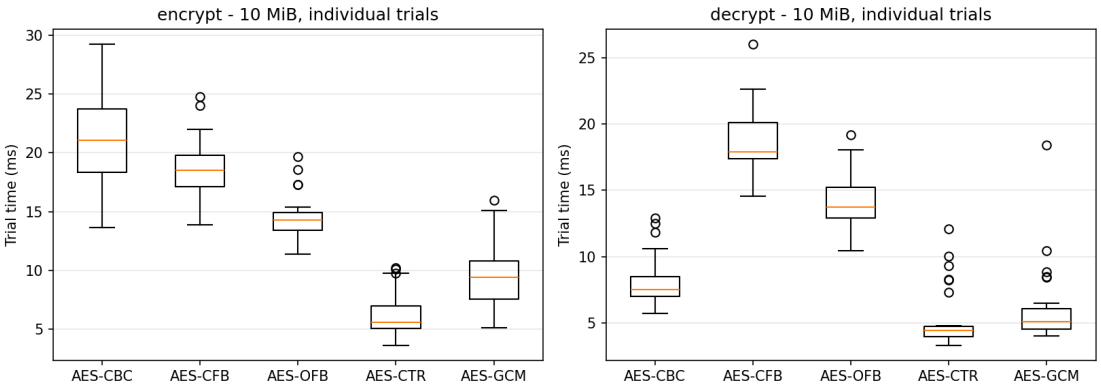
Mean throughput (MiB/s)	1 KiB	64 KiB	1 MiB	10 MiB	50 MiB
AES-CBC	58	1144	1926	1218	368
AES-CFB	71	488	637	533	376
AES-OFB	72	366	840	698	435
AES-CTR	80	1525	2845	1920	721
AES-GCM	41	1071	2777	1661	475
DES-CBC	19	24	31	27	25
RC4	77	378	395	343	278
fastest required mode	**AES-CTR**	**AES-CTR**	**AES-CTR**	**AES-CTR**	**AES-CTR**



Throughput versus size



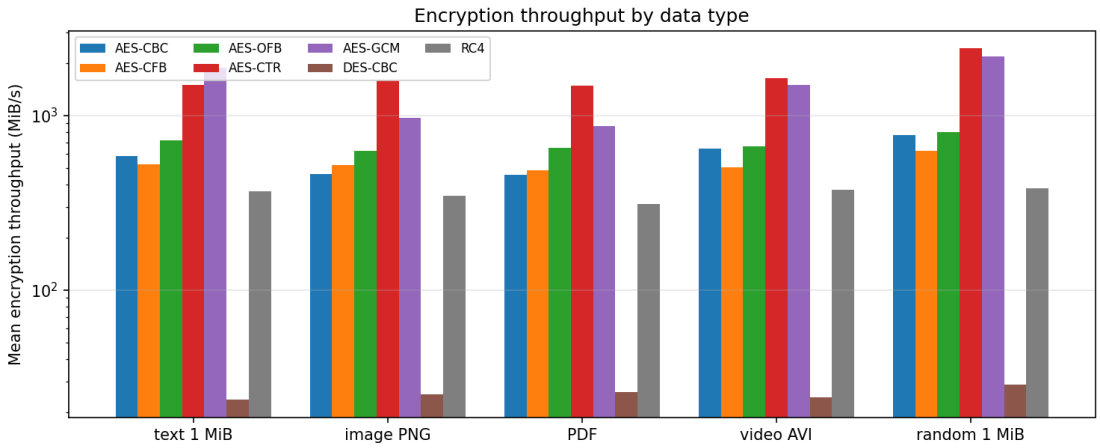
Mean time at 10 MiB with 95 % CI



Distribution of individual trials, 10 MiB

By data type (mean encryption throughput; synthetic stand-ins for text, image, PDF and video):

Encryption MiB/s	text 1 MiB (1024 KiB)	PNG image (1655 KiB)	PDF (546 KiB)	AVI video (450 KiB)	random 1 MiB (1024 KiB)
AES-CBC	584	461	455	645	769
AES-CFB	525	518	485	506	628
AES-OFB	720	625	653	667	806
AES-CTR	1491	1593	1483	1631	2417
AES-GCM	1877	964	869	1489	2172
DES-CBC	23	25	26	24	29
RC4	368	345	311	376	382



Throughput by data type

10.3 File I/O measured separately

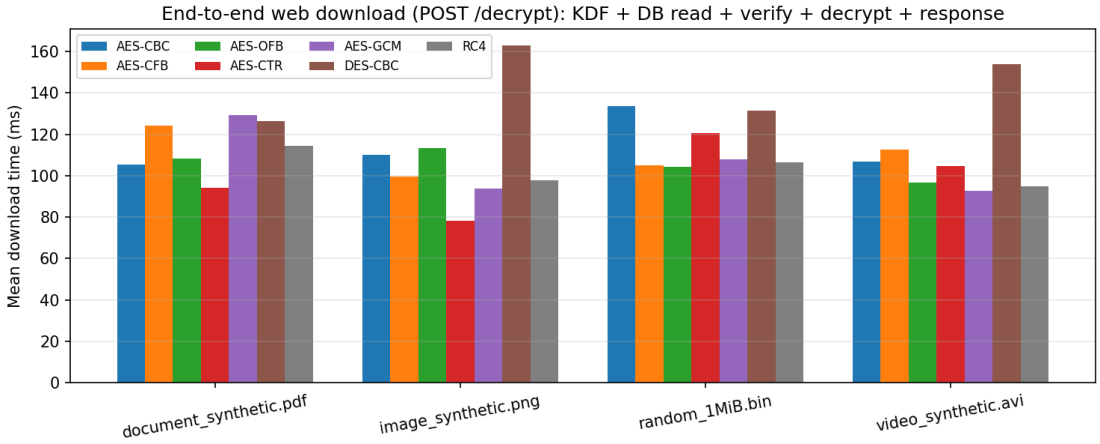
Plain `read_bytes()` and `write_bytes()` on the same inputs (30 trials, page-cache effects included, so these are optimistic):

File	Size (B)	Read mean (ms)	Write mean (ms)
random_1KiB.bin	1024	0.035	0.295
random_64KiB.bin	65536	0.196	1.023
video_synthetic.avi	460954	0.165	0.524
document_synthetic.pdf	559953	0.120	0.586
text_1MiB.txt	1048576	0.198	1.065
random_1MiB.bin	1048576	0.169	0.835
image_synthetic.png	1695017	0.279	1.119
random_10MiB.bin	10485760	2.228	4.748
random_50MiB.bin	52428800	32.577	28.884

10.4 End-to-end web download through the Flask app

`POST /decrypt/<id>` through the real application with the production setting of 600,000 PBKDF2 iterations (10 downloads per cell; each response was verified byte-for-byte):

Mean download (ms)	document_synthetic.pdf	image_synthetic.png	random_1MiB.bin	video_synthetic.avi
AES-CBC	105.2	110.0	133.5	106.8
AES-CFB	124.2	99.5	105.1	112.7
AES-OFB	108.4	113.5	104.3	96.8
AES-CTR	94.0	78.4	120.6	104.7
AES-GCM	129.2	93.7	108.1	92.7
DES-CBC	126.3	162.8	131.3	153.7
RC4	114.5	97.7	106.6	94.7



End-to-end download

The password KDF alone, measured separately (20 runs), took a mean of **101 ms** (median 109, SD 15), while all end-to-end downloads had a median of 109 ms (range 73–203 ms). So for files of the sizes tested here the download latency is **dominated by the KDF, which is identical for every algorithm**; differences between the AES modes and RC4 in this table are mostly within noise. DES-CBC is visibly slower for the larger files (e.g. the PNG and AVI rows), where its slow cipher speed is no longer hidden by the KDF.

10.5 Observations (what the data support)

- AES-CTR had the lowest mean encryption time among the four required modes at: 64 KiB, 1 MiB, 10 MiB, 50 MiB. At 10 MiB its mean encryption time was 0.30x that of CBC, 0.34x CFB and 0.44x OFB (95 % CIs do not overlap against CBC).
- CBC encryption cannot be parallelised (each block depends on the previous ciphertext block) whereas CBC decryption can; at 1 MiB CBC decryption took 0.40x its encryption time, and at 10 MiB 0.39x.
- OFB keystream generation is sequential in both directions, and CFB decryption (which could be parallelised in principle) was not observed to be faster than CFB encryption here; their decryption/encryption time ratios at 10 MiB were close to 1 (CFB 1.01x, OFB 0.99x decrypt/encrypt).
- AES-GCM, which also authenticates, was as fast as or faster than CBC/CFB/OFB here (0.45x CBC at 10 MiB, encryption); a stream-style mode with hardware support can be

cheaper than a mode without integrity protection, so speed alone says little about security.

- DES-CBC was about **19x slower** than AES-CBC at 10 MiB (software 3DES-based path, no hardware acceleration) and, from 1 MiB upward, RC4 was slower than every AES mode tested on this machine (it was the fastest row only at 1 KiB, where setup cost dominates): “old and simple” did not mean “fast”.
- Small inputs (1 KiB) are dominated by per-call setup cost: the four required modes’ mean encryption times were all between 59 and 70 microseconds with SDs of 10–18 µs, so they are not reliably distinguishable there. Throughput peaked around 1 MiB and fell at 10–50 MiB; a likely (untested) cause is cache/memory effects and allocation of large buffers.
- Throughput by data type did not follow a consistent ordering by content type. For one mode it still varied by up to about a factor of two (AES-GCM: 869–1877 MiB/s across text, PNG, PDF and AVI), but those files also differ in size (0.4–1.6 MiB) and the machine is noisy, so this experiment cannot attribute differences to the content type. Block-cipher speed is not expected to depend on plaintext content.

11. Ciphertext comparison and technical interpretation

Lab experiments use a **fixed lab key and IV only for reproducibility** (`benchmark/ciphertext_comparison.py`); they must never be reused for real data. Plaintext: first 4096 bytes of the text fixture.

(a) Different modes give different ciphertexts. Each mode combines key, IV and plaintext differently (CBC XORs the previous ciphertext block before encrypting; CFB/OFB/CTR XOR plaintext with a keystream; GCM is CTR plus a tag). With the same key and IV, pairs of ciphertexts differed in 49.6 %–50.6 % of bits (≈ 50 %, as for unrelated random strings). One legitimate coincidence: with the *same* IV, CFB, OFB and CTR produce an **identical first block** (AES-CFB vs AES-CTR, AES-CFB vs AES-OFB, AES-OFB vs AES-CTR), because each XORs plaintext block 1 with E_K(IV); they diverge from block 2 on. (This is also why reusing an IV is dangerous.)

(b) Randomness controls reproducibility. With identical key *and* IV, every mode was deterministic (all `True` in `ciphertext_comparison.json`). With a fresh random IV, ciphertexts of the same plaintext differed (AES-CBC: 50.0% bits, AES-CFB: 50.6% bits, AES-CTR: 50.1% bits, AES-GCM: 49.7% bits, AES-OFB: 50.0% bits). In the app the salt and IV are also fresh per file, so even the same file under the same password gives different stored bytes; an observer cannot see that two files are equal. Reproducibility therefore requires fixing key and IV, which is only acceptable in experiments.

(c) Size and padding.

Plaintext length (B)	0	1	15	16	17	31	32	1000
AES-CBC	16	16	16	32	32	32	48	1008
AES-CFB	0	1	15	16	17	31	32	1000
AES-CTR	0	1	15	16	17	31	32	1000
AES-GCM	16	17	31	32	33	47	48	1016
AES-OFB	0	1	15	16	17	31	32	1000

CBC/DES pad with PKCS#7: 1–16 extra bytes (a full extra block when the length is already a multiple of 16; even empty input becomes one block). CFB/OFB/CTR/RC4 keep the exact length; GCM adds a 16-byte tag. The app additionally stores a 16-byte salt, an IV and (except GCM) a 32-byte HMAC per file.

(d) Why ECB is excluded. Eight identical 16-byte plaintext blocks produced duplicate ciphertext blocks only under ECB (demonstration only): AES-CBC: 0, AES-CFB: 0, AES-CTR: 0, AES-ECB (demonstration only, NOT used): 7, AES-GCM: 0, AES-OFB: 0. Equal plaintext blocks leak structure in ECB.

(e) Bit-error behaviour (raw, unauthenticated modes).

Mode	Result of flipping 1 ciphertext bit in block 2	Wrong plaintext bits per block (blocks 0-9)
AES-CBC	decrypted with errors	[0, 0, 60, 1, 0, 0, 0, 0, 0, 0]
AES-CFB	decrypted with errors	[0, 0, 1, 71, 0, 0, 0, 0, 0, 0]
AES-CTR	decrypted with errors	[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
AES-GCM	decryption rejected (authentication/padding failure)	-
AES-OFB	decrypted with errors	[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]

CTR and OFB: only the flipped bit changes (an attacker can flip chosen plaintext bits: malleability). CBC: the whole corresponding block is garbled and one bit flips in the next block. CFB: the corresponding bit flips and the following block is garbled. AES-GCM rejects the data. In the app the HMAC / GCM tag catches all of these **before** any plaintext is released, so error propagation is only visible in the raw modes.

(f) Limits of speed-only comparisons. Speed depends on the library, CPU instructions, buffer sizes and noise; it ignores security properties: none of CBC/CFB/OFB/CTR provides integrity on

its own, CBC padding can enable padding-oracle attacks if error messages differ (the app uses one generic error and verifies a MAC first), CTR/OFB/GCM are catastrophic under nonce reuse, and CFB/CBC leak IV-related structure when IVs are predictable. A mode must be chosen for its security and fitness first; AES-GCM (or encrypt-then-MAC) is the modern choice.

12. Functional testing results

Executed with `python -m unittest discover -s tests -t . -v` on the environment of Section 9; full log in `report/test_results.txt`.

Result: 69 tests run, OK. (`test_crypto`: 22, `test_webapp`: 37, `test_benchmark`: 10; failures: 0.)

#	Required area	Where tested
1–2	Registration, login, invalid logins, throttling, logout	`test_webapp.Auth`
3–5	Upload + encryption with AES (all modes), RC4, DES, and `ALL`	`UploadDownload.test_each_algorithm_roundtrip_byte_for_byte`, `test_all_three_required_algorithms_option`
6	Byte-for-byte integrity	same tests (`assertEqual(r.data, data)`)
7	Wrong password	`UploadDownload.test_wrong_password_returns_no_file`, `test_crypto.Tamper`
8	Empty files, arbitrary binary	web: empty rejected by policy; crypto: sizes 0…100 003 B for every algorithm (`RoundTrip`)
9	Image, document, video	`test_all_file_types_and_categories` (png, jpg, jpeg, pdf, doc, docx, xls, xlsx, mp4, mov, avi, webm, mkv), `test_large_video_like_upload`
10	Unsupported types, oversized	`test_unsupported_and_mismatched_types`, `test_oversized_upload`
11	Corrupted / tampered ciphertext	`Tampering.*`, `test_crypto.Tamper.*`
12	Missing / invalid metadata	`test_invalid_or_missing_metadata`,

#	Required area	Where tested
		`test_null_metadata_blocked_by_schema`, `Tamper.test_invalid_metadata`
13	Unauthorised access	`Authorization.*`
14	Path traversal	`PathTraversal.*`
15	Persistence / restart	`Persistence.test_survives_application_restart`
16–17	Every required AES mode, varied sizes incl. empty	`test_crypto.KnownAnswer` (NIST SP 800-38A vectors), `RoundTrip`, `test_benchmark.Reproducibility`
18	Benchmark output completeness / reproducibility	`test_benchmark.Runner`, `Fixtures`, `Reproducibility`

Bugs found and fixed during this work include: random per-start `SECRET_KEY`; `debug=True`; plaintext written to `uploads/` with the user-controlled name (path traversal / overwrite); padding never validated and no integrity check, so tampered or wrong-password CBC/RC4 data could be returned as a "successful" download; GET logout (CSRF); no upload or password validation; and during testing, a template bug that displayed blank filenames and a sanitiser that left `..` inside names. The tests do **not** cover a real browser, TLS, concurrency, or very large (≥ 1 GB) files. Not executed: `pytest` itself (not installable offline; the suite is plain `unittest`, which `pytest` also collects).

13. Security limitations and ethical considerations

- **Not production-secure and not legally certified.** Technical controls help with GDPR Art. 5 principles (integrity/confidentiality, data minimisation, storage limitation via deletion), Art. 25/32 (security by design, appropriate measures) and UU PDP duties to protect personal data, but compliance also needs lawful basis and consent management, privacy notices, retention policy, breach procedures, records of processing, access logging, data-subject rights tooling (e.g. account erasure/export), and for GDPR possibly a DPIA. None of these are implemented beyond what is listed here; no legal compliance is claimed.
- DES and RC4 are broken (56-bit keys; biased keystream; 64-bit block issues such as Sweet32). They are present only because the assignment requires them; do not use them for real data.
- Security depends on the **strength of the per-file password** (PBKDF2 slows guessing but cannot save "12345678"); there is no recovery: a forgotten password means permanent data loss.

- The server sees the file password and plaintext during upload/download (server-side encryption). End-to-end/client-side encryption would remove that trust.
- Plaintext metadata (name, size, time, category); login throttle held in process memory; no 2FA; CDN assets for Bootstrap/fonts; whole-file in-memory processing; magic-byte checks are not malware scanning.
- Ethics: use only synthetic or consented data when testing. The repository contains no real IDs, documents or credentials; benchmark fixtures are synthetic.

14. Justification of major development decisions

Decision	Reason
Rebuild on `cryptography`, `sqlite3`	Required packages (PyCryptodome, Flask-SQLAlchemy) could not be installed offline; stdlib + one well-known library means fewer dependencies and a smaller attack surface.
Ciphertext as SQLite BLOB	Literally "stored in a database"; transactional; no filesystem paths to attack. Cost: memory use and size limit.
Encrypt-then-MAC, HKDF-separated keys	Unauthenticated modes return garbage or enable attacks; the slide demands a bug-free program, so tampering must never yield "successful" output.
PBKDF2 600 000 iterations (kept from original)	This matches the figure OWASP's Password Storage Cheat Sheet gave for PBKDF2-HMAC-SHA256 when last checked (re-verify the current value before deployment); ≈ 101 ms here. Iteration count stored per file for future migration.
AES-CBC default for the app	Satisfies the "non-ECB mode" requirement and was the original design; CFB/OFB/CTR/GCM selectable. GCM recommended for real use.
`ALL` option	Reasonable reading of "encrypted with all of these algorithms"; flagged for lecturer confirmation.
Generic error messages, 404 for foreign files	Avoid oracles and enumeration.
Separate cipher-only, I/O and end-to-end timings	The slide says "download multiple times", which includes KDF/DB/HTTP costs that would otherwise mask cipher differences.
Synthetic, seeded fixtures	Reproducible, no personal data, portable.

15. Conclusion

The application stores ID-card images, documents and videos as ciphertext in the database, encrypted per file with AES (CBC, CFB, OFB, CTR or GCM), DES or RC4, returns them byte-for-byte after successful verification, and isolates each user's data. In the measurements on this machine, AES-CTR (and the CTR-based GCM) were the fastest at sizes from 1 MiB, CBC encryption was slower than CBC decryption at 1 and 10 MiB, CFB and OFB decryption was no faster than their encryption, and DES was roughly 19 times slower than AES-CBC at 10 MiB. Ciphertexts differ between modes and between runs whenever the IV, nonce or salt differ, CBC pads and the stream-like modes do not, and bit errors propagate differently in each mode, which is a security consideration that a pure speed ranking ignores. For real end-to-end download times, the password KDF dominated all algorithm differences for the file sizes tested. Limitations: single noisy machine, single run, synthetic data, ambiguity in two assignment requirements (to be confirmed), and non-production scope.

16. References

1. NIST SP 800-38A, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques* (CBC, CFB, OFB, CTR). https://csrc.nist.gov/pubs/sp/800/38/a/final
2. NIST SP 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*. https://csrc.nist.gov/pubs/sp/800/38/d/final
3. NIST FIPS 197, *Advanced Encryption Standard (AES)*. https://csrc.nist.gov/pubs/fips/197/final
4. RFC 7465, *Prohibiting RC4 Cipher Suites*. https://datatracker.ietf.org/doc/html/rfc7465 ; RFC 6229, *Test Vectors for the Stream Cipher RC4*. https://datatracker.ietf.org/doc/html/rfc6229
5. RFC 8018, *PKCS #5: Password-Based Cryptography Specification v2.1* (PBKDF2). https://datatracker.ietf.org/doc/html/rfc8018 ; RFC 5869, *HKDF*. https://datatracker.ietf.org/doc/html/rfc5869
6. pyca/cryptography documentation. https://cryptography.io/en/latest/
7. OWASP Cheat Sheet Series: Password Storage; Cross-Site Request Forgery Prevention. https://cheatsheetseries.owasp.org/
8. Flask documentation. https://flask.palletsprojects.com/ ; Werkzeug security utilities. https://werkzeug.palletsprojects.com/
9. SQLite documentation (data types, limits). https://www.sqlite.org/limits.html
10. Regulation (EU) 2016/679 (GDPR), Arts. 4, 5, 9, 25, 32. https://eur-lex.europa.eu/eli/reg/2016/679/oj
11. Undang-Undang Republik Indonesia Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi, Art. 4 (official text, JDIH Kemenkeu): https://jdih.kemenkeu.go.id/download/46c17b53-4a45-411c-9b76-c6799e5769fc/UU_Nomor_27_Tahun_2022 Perlindungan Data Pribadi.pdf
12. Bhargavan, K., Leurent, G., *On the Practical (In-)Security of 64-bit Block Ciphers (Sweet32)*, ACM CCS 2016. https://sweet32.info/
13. Bellare, M., Namprempre, C., *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000.

URLs were not all re-fetched in the build environment; the UU PDP Art. 4 categories were checked against secondary summaries and the official-text search result, so confirm article wording against the official text before submission.